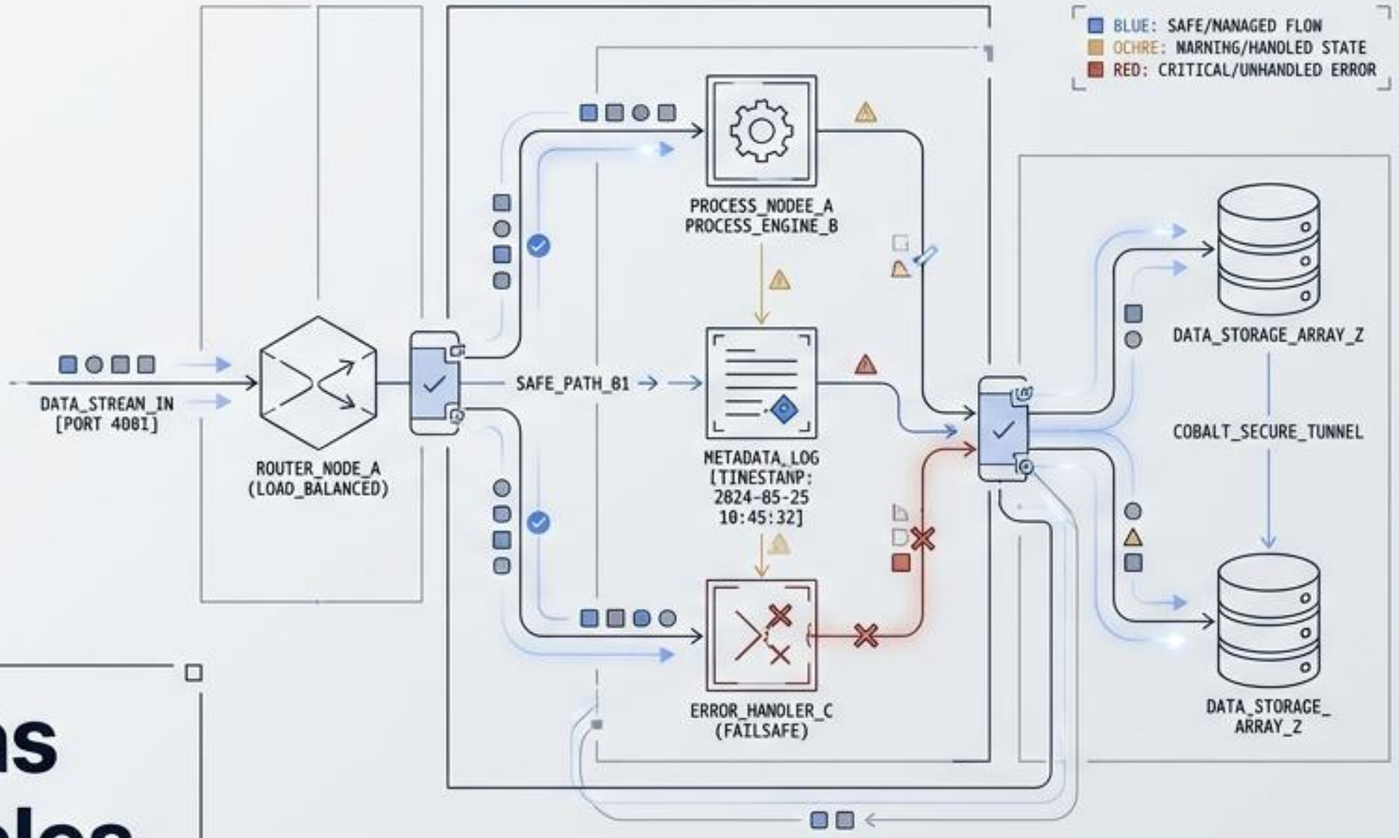
Construyendo Sistemas Resilientes y Observables

Semana 5: Manejo de Errores y Excepciones | Construcción de Software II

Universidad Nacional de Ingeniería (UNI)



NODE ID	STATUS	LATENCY (ms)	ERROR RATE (%)	THROUGHPUT (req/s)	METADATA HASH
NODE_A	ACTIVE [COBALT]	45.2	0.01	1590	0x4F92A3C8
NODE_B	WARNING [OCHRE]	129.5	1.25	809	6x7B106E2F
NODE_C	CRITICAL [RED]	N/A	100.0	0	6x00000000

```
[INFO] 2024-05-25 10:45:30 - System initialization complete. [COBALT]
[WARN] 2024-05-25 10:45:35 - High latency detected in NODE_B. [OCHRE]
[ERROR] 2024-05-25 M:45:40 - Connection refused to NODE_C, initiating failover. [RED]
[INFO] 2824-85-25 10:45:42 - Fallover successful. Data rerouted. [COBALT]
```

Gemini Notebook

De la Prevención a la Resiliencia en Tiempo de Ejecución



Programación Defensiva

La barrera defensiva no es suficiente: La semana pasada construimos barricadas mediante contratos para rechazar datos inválidos.



Manejo de Excepciones y Tolerancia a Fallas

La inevitabilidad de la falla: ¿Qué sucede cuando la red colapsa o la base de datos no responde?
El objetivo hoy es transicionar a construir sistemas que sobrevivan y expliquen claramente por qué colapsaron.

Gemini Notebook

El Diagnóstico Fundamental: Condición de Error vs. Excepción

	Condición de Error	Excepción (Tiempo de Ejecución)
Previsibilidad	Esperada y predecible (ej. "Contraseña muy corta"). Es parte de las reglas del negocio.	Inesperada o anomalía ambiental (ej. "Timeout en DB", 'Puntero Nulo").
Mecanismo de Control	Manejada explícitamente mediante tipos de retorno (tuplas, objetos Result).	Abandona el bloque actual, desenrolla la pila de llamadas buscando un manejador.
Responsabilidad de Resolución	El cliente o usuario final puede corregir el problema directamente.	Requiere intervención del desarrollador o administrador (SRE).

Gemini Notebook

Catálogo de Estrategias frente a Condiciones de Error



Gemini Notebook

Políticas de Excepciones: El Árbol de Decisión

¿Se espera razonablemente que el código que llama pueda recuperarse de esta falla?

SÍ

Excepciones Comprobadas (Checked)

- Contingencias ambientales (ej. `FileNotFoundException`).
- El compilador obliga al desarrollador a manejarlas (try-catch) o declararlas.

NO

Excepciones No Comprobadas (Unchecked / Runtime)

- Errores de programación (ej. `NullPointerException`).
- Mentalidad *Fail-fast*. Revelan un defecto lógico que debe ser corregido, no manejado.

Gemini Notebook

Diseño de Jerarquías de Dominio

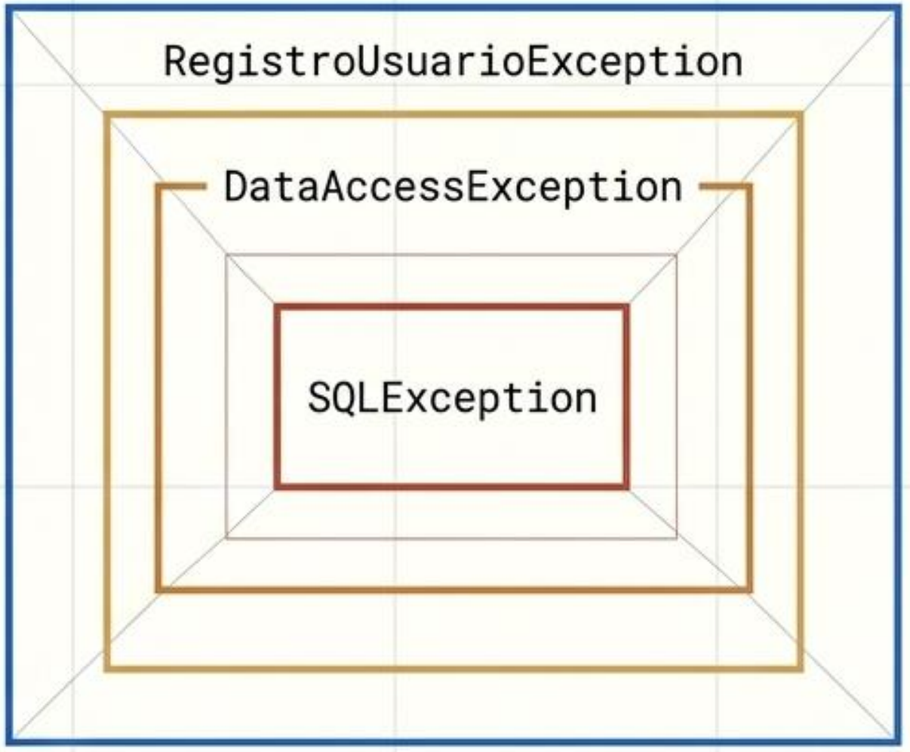


Gemini Notebook

Envoltura y Relanzamiento (Exception Chaining)

El Problema: Las capas superiores no deben conocer detalles técnicos de la base de datos, pero no podemos perder la causa original (Root Cause) para el diagnóstico.

La Solución: Capturar la excepción de bajo nivel, envolverla en una de dominio y relanzarla.




```
try {
    repositorio.guardar(usuario);
} catch (SQLException e) {
    // Envoltura preservando la causa original 'e'
    throw new RegistroUsuarioException("Falla al registrar usuario en el sistema", e);
}
```

Gemini Notebook

Antipatrones Vol. 1: El Agujero Negro del Diagnóstico

El Código Peligroso

1. Swallowing (Bloque catch vacío)

```
try {
    procesarPago();
} catch (Exception e) {
    // nada
}
```

2. Catch-All Genérico

```
catch (Exception e) {
    ...
}
```

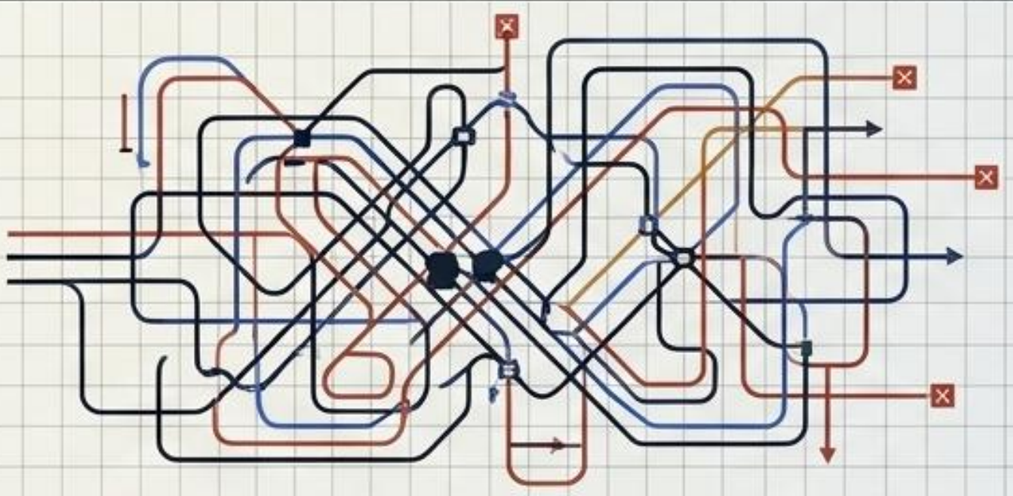
La Realidad Catastrófica



- **Silencio Mortal**: Tragar excepciones oculta fallas críticas. El sistema continúa en un estado inconsistente sin dejar rastro en los logs.
- **Captura Excesiva**: Usar Exception genérico captura errores irrecuperables (como OutOfMemoryError), **impidiendo que el servicio se reinicie.**

Gemini Notebook

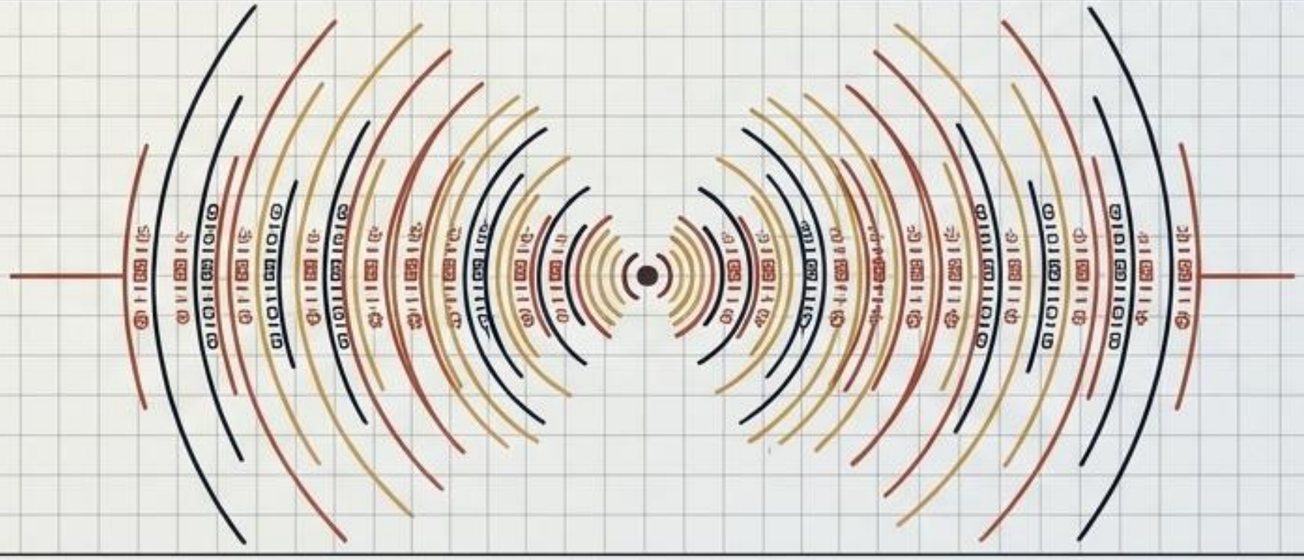
Antipatrones Vol. 2: Control de Flujo y Ruido de Logs



Control de Flujo

Antipatrón: Usar throw y catch como if/else.

Rompe la legibilidad y destruye el rendimiento. Generar el stack trace es una de las operaciones más costosas de la JVM.



Log-and-Throw

```
catch (Exception e) {
  log.error(e);
  throw e;
}
```

Crea múltiples registros idénticos en diferentes capas. Ensucia los logs e incrementa costos de almacenamiento.

Gemini Notebook

Diseño de Mensajes de Excepción Informativos

Principio Clave: El mensaje de una excepción es para el ingeniero **que depura**, no para el usuario final.

Antes (Inútil - Falta Contexto)

```
throw new OrderException("No se pudo procesar la orden.");
```

¿Qué orden? ¿Qué usuario? ¿Estado?

Después (Grado Producción - Rico en Contexto)

```
throw new OrderException(String.format(
      "Falla format("Falla al procesar Orden [ID: %s]
      para Usuario [ID: %s]. Estado actual: %s", orderId,
      userId, userId,
      status));
```

Incluir el estado exacto de las variables en el momento del colapso ahorra horas de depuración.

Gemini Notebook

Registro Estructurado (Structured Logging) como Técnica de Construcción

Texto No Estructurado

```
[2024-10-25 14:32:01] ERROR - Error procesando pago id=994 userId=4589 error=Timeout
```

Objeto JSON Estructurado

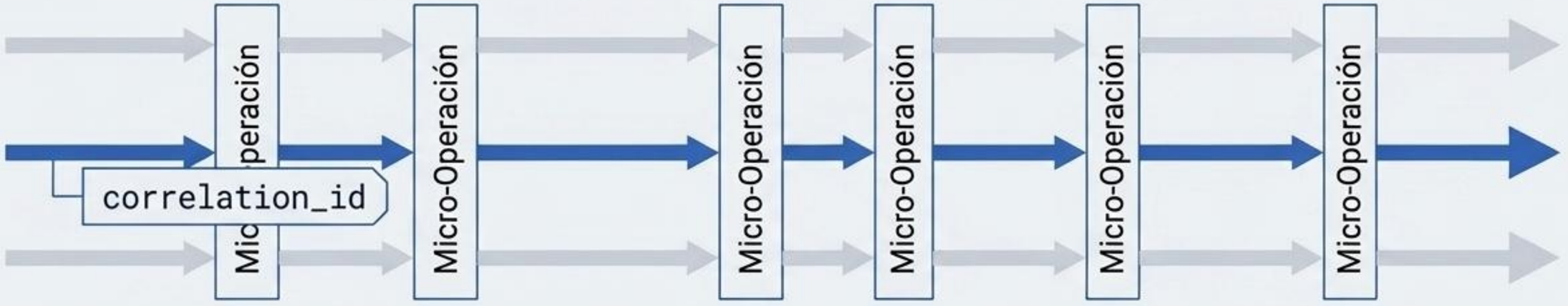
```
{
  "timestamp": "2024-10-25T14:32:01Z",
  "level": "ERROR",
  "event": "payment_processing_failed",
  "order_id": 994,
  "user_id": 4589,
  "reason": "timeout_database"
}
```

Los logs ya no son texto para humanos. Son conjuntos de datos consultados por máquinas. Separar el mensaje de los metadatos es vital.

Gemini Notebook

Contexto de Diagnóstico Mapeado (MDC & Correlation_ID)

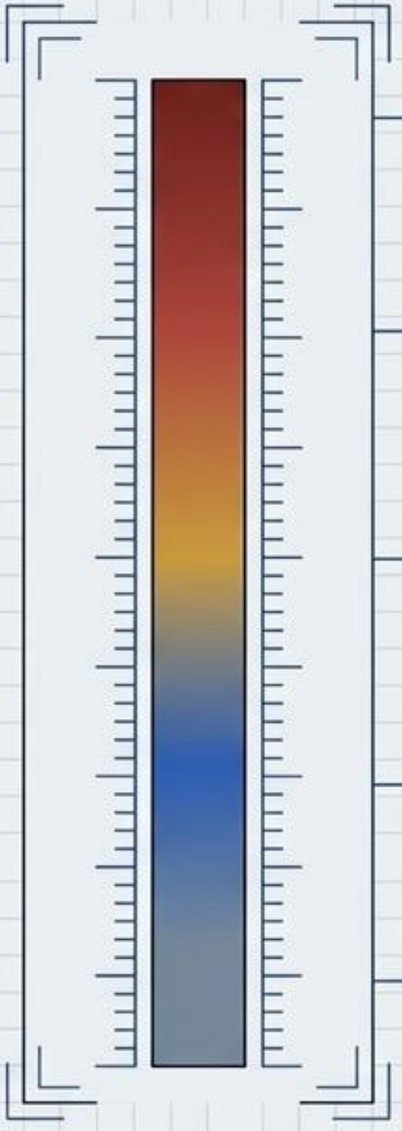
Threading the Needle



El Problema	MDC (Mapped Diagnostic Context)	El Hilo Conductor
En un servidor, cientos de hilos escriben logs simultáneamente. Leer un log secuencial es imposible.	Un espacio de memoria específico del hilo (Thread-Local) para inyectar contexto (ej. user_id).	El Correlation ID asegura que cada línea de log de ese hilo incluya este ID automáticamente.

Gemini Notebook

Taxonomía de Niveles de Log en Producción



FATAL
Colapso del sistema. Despierta al ingeniero de guardia.

ERROR
Falla en una operación. Un usuario fue afectado. Dispara alertas.

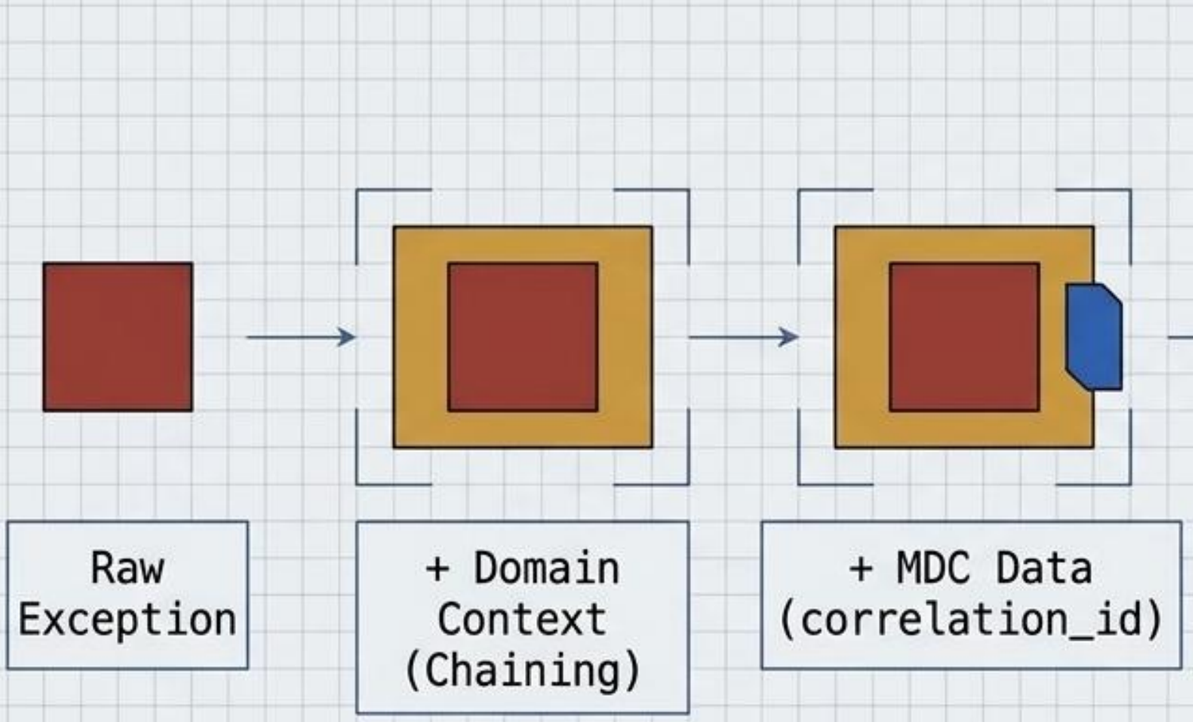
WARN
Anomalías recuperables. Requiere atención eventual, pero la operación fue exitosa.

INFO
Hitos de negocio. Indica que el sistema está saludable.

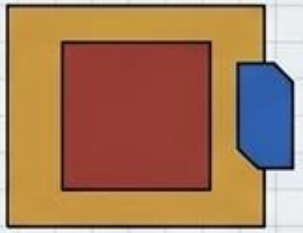
TRACE / DEBUG
Exclusivo para desarrollo. Detalles microscópicos del flujo.

Gemini Notebook

Síntesis: El Artefacto de Falla



JSON LOG DASHBOARD



```
{
  "correlation_id": "abc12345",
  "level": "ERROR",
  "message": "Domain failure",
  "exception": "NullPointerException",
  "context": {
    "context": {...}
  }
}
```

"Una excepción no es solo un salto de control elegante. Diseñada correctamente, es un artefacto de diagnóstico telemétrico: una cápsula rica en contexto diseñada para viajar por las capas arquitectónicas y documentar su propio colapso de forma estructurada.

Gemini Notebook

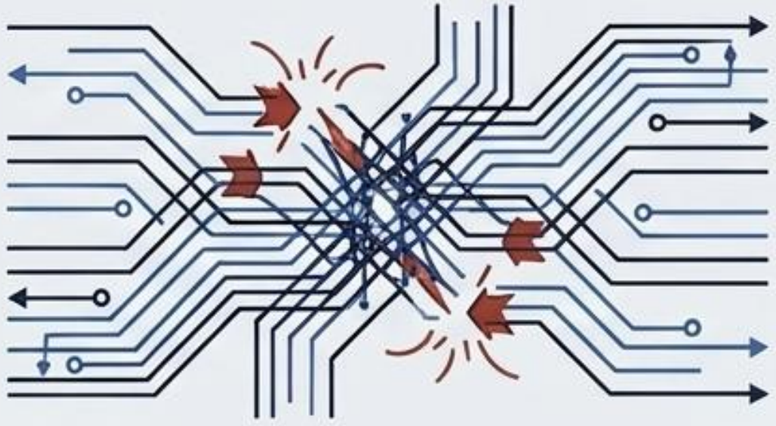
Resumen y Siguiente Frontera: Concurrencia

Dominio de la Semana 5



- Diferenciamos errores predecibles de excepciones críticas.
- Diseñamos jerarquías encapsuladas (Exception Chaining).
- Erradicamos antipatrones de diagnóstico.
- Instrumentamos código para máquinas (Structured Logging).

Próxima Semana: Concurrencia



Hoy vimos cómo el MDC usa almacenamiento específico por hilo (Thread-Local). Pero, ¿qué sucede cuando múltiples hilos intentan modificar el mismo espacio de memoria a la vez?

Semana 6: Condiciones de carrera, sincronización y seguridad de hilos.

Gemini Notebook